

“产品”正在变成能力发现过程

Tibo x TechCrunch：ChatGPT Work、最小界面与智能成本

编辑整理的 Q&A 访谈 • 正式文字专访 • 会谈日期 2026.08.25

一句话结论

这篇专访补上了产品与商业视角：OpenAI 的目标不是给 Agent 叠更多按钮，而是让界面尽量退后，让模型能力自然表达；产品团队一边推进 frontier，一边从真实使用中“发现”新的产品形态，同时持续压低单位智能成本。

四个最值得关注的信号

01 Diffusion Codex 先服务“宽容的技术用户”，能力成熟后再向更广泛知识工作扩散。	02 Minimal surface 界面应该尽量简单、自然，让用户不必学习复杂应用操作。
03 Discovery as product 模型能力边界在变化，产品 roadmap 也要通过真实使用持续发现，而不是一次性定义完。	04 Cost curve 长期目标是同样一美元买到越来越多 utility；效率提升是 Agent 普及的经济前提。

说明：下文将“访谈观点”“技术解读”“方向推演”分开呈现；预测不视为正式产品承诺。

OVERVIEW

访谈背景与章节地图

受访者：Thibault “Tibo” Sottiaux，OpenAI Core Product / Codex 相关负责人。

对谈方：Tim Fernholz（TechCrunch）

形式：编辑整理的 Q&A 访谈；**时长：**正式文字专访

- 访谈观点
- 技术解读
- 方向推演
- 外部核验

章节地图 / 讨论顺序

时间 / 顺序	主题
Q1	Tibo 的职责：API、Agent infra、Enterprise、ChatGPT、Codex
Q2	为什么 ChatGPT Work 重要：把 coding-agent 能力带给所有知识工作者
Q3	AI diffusion 与赢得非技术用户
Q4	Model lead 还是 UI lead：为什么偏向 minimal product surface
Q5	“世界准备好了吗”：产品简单但不牺牲能力
Q6	Discovery：先发现模型变强后能做什么，再围绕能力做产品
Q7	智能成本：效率提升与单位 utility 成本下降
Q8	连接邮件 / 消息后的安全担忧

阅读方法

先看 Tibo 明确表达的系统设计原则，再看这些原则对 Agent Runtime、Serving、Sandbox、Memory、Multi-agent 或软件工程流程的含义。不要把方向判断直接当作已经落地的功能。

CORE IDEAS

1. “Discovery as product design” 很反传统

- 传统软件先定义需求，再设计功能和 UI；frontier AI 产品的麻烦是底层能力每几个月都在变，团队自己也不能提前完整知道新模型会在哪些任务突然跨过可用阈值。
- Tibo 的做法更像：推出能力 -> 看真实用户把它拿去做什么 -> 找到高价值、可重复的模式 -> 再把这些模式包装成更简单的产品。
- 这使 eval / telemetry / user feedback 不只是“发布后的监控”，而是产品发现引擎。

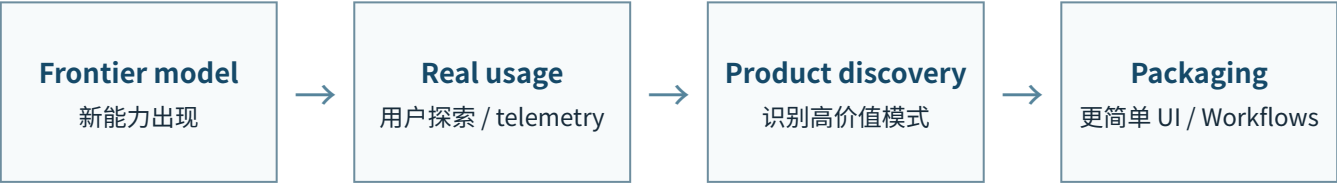
核心判断：这篇专访补上了产品与商业视角：OpenAI 的目标不是给 Agent 叠更多按钮，而是让界面尽量退后，让模型能力自然表达；产品团队一边推进 frontier，一边从真实使用中“发现”新的产品形态，同时持续压低单位智能成本。

2. Minimal surface: 最强 UI 可能是“少做 UI”

- Tibo 倾向让模型直接表达能力，避免用户学习复杂按钮。Voice 是这个方向的自然延伸：人用本来的交流方式表达目标，系统适配人。
- 但越少显式 UI，系统越需要在后台做好：意图识别、权限确认、可逆操作、进度可见性、结果验证。否则“魔法”只会变成不可预测。
- 因此 minimal surface 不是工程更少，而是把复杂度从前台搬到 Agent runtime 和安全层。

SYSTEM VIEW

把访谈观点翻译成系统结构



这里最容易误解的一点

Agent 的“能力”并不等于裸模型能力。对真实任务而言，更接近：

$$\text{Agent Utility} = \text{Model} \times \text{Context} \times \text{Tools} \times \text{Runtime} \times \text{Safety} \times \text{Verification}$$

任何一项接近 0，端到端体验都会坍塌；这也是为什么模型 benchmark 提升并不总能等比例转化为产品收益。

工程上应该关注的闭环

Observe	任务 trace、tool call、权限、失败位置、人工介入、最终结果。
Diagnose	区分是模型、Harness、环境、数据、延迟还是安全策略导致失败。
Improve	把问题放回最合适的层修，而不是默认继续堆 prompt 或规则。
Verify	用可重复 eval 证明端到端成功率、成本和 latency 真正改善。

AI INFRA TAKEAWAYS

对产品 Infra / Unit Economics 的启发

Telemetry	需要 task-level taxonomy：用户到底委派了什么、成功率、人工修改量、最终 artifact 是否被使用。
Cost per utility	对 Agent 更合理的单位是 \$/completed-task、tokens/success，而不是 \$/M token。
Adaptive compute	不同任务自动选模型、reasoning effort、并发数和工具，才能把 frontier 能力做进大众价格。
Progressive disclosure	UI 简洁但高风险能力要按需暴露权限和解释，避免“一键全权访问”。
Feedback flywheel	真实使用失败要能回流到 eval set、Harness 修复和模型训练，形成闭环。

给组内讨论的一个统一框架

把每项 Agent 优化问成四个问题：①端到端成功率提升多少；②wall-clock 缩短多少；③总算力/Token/工具成本增加多少；④风险与人工验收成本是否变高。单看模型分数、Kernel 峰值或单 Token 价格都不够。

$$T_{total} = T_{queue} + T_{prefill} + T_{decode} + T_{tool} + T_{network} + T_{environment} + T_{verification} + T_{human_wait}$$

WHAT TO BELIEVE / WHAT TO DO

需要打折看的部分

- 这是公司核心产品负责人接受媒体采访，关于 adoption、价值和成本趋势的表述带有产品立场。
- 文章为 “lightly edited and condensed”，并非完整逐字稿；不宜对缺失上下文做过度推断。
- 价格、计划和产品名称变化很快，本报告只分析访谈中的战略逻辑，不把当时的套餐细节当长期事实。

给组员的三个可执行动作

1. 把团队 Agent 项目的核心 KPI 从 token throughput 扩展到 cost/successful-task 和 human-edit-minutes。
2. 每次模型升级都做 “capability discovery sprint”：让真实用户自由探索，再把新用例固化成 eval。
3. 设计 minimal UI 时同步列出后台必须补齐的安全 / 可观察 / 恢复机制。

本场会谈的最终一句话

这篇专访补上了产品与商业视角：OpenAI 的目标不是给 Agent 叠更多按钮，而是让界面尽量退后，让模型能力自然表达；产品团队一边推进 frontier，一边从真实使用中“发现”新的产品形态，同时持续压低单位智能成本。

原始资料

<https://techcrunch.com/2026/08/25/the-world-seems-to-be-ready-an-interview-with-openai-head-of-product-thibault-sottiaux/>

整理日期：2026-09-10。报告为技术阅读笔记，不代表 OpenAI 官方路线承诺；产品状态以最新官方文档为准。